



本科生实验报告

实验课程：编译原理实验

实验名称：手工编写递归下降预测分析程序

专业名称：计算机科学与技术

实验地点：东校园实验中心大楼 B202

报告时间：2026/6/7

学号	23320143
姓名	唐雅妍
邮箱	tangyy77@mail2.sysu.edu.cn

目录

1	实验目的	2
2	设计 Oberon-0 语言的翻译模式	2
3	FIRST / FOLLOW 集计算	3
4	递归下降分析器设计	3
4.1	整体架构	3
5	流程图生成	5
5.0.1	语句序列栈	5
5.1	过程嵌套时的状态恢复	6
5.2	代码串生成	7
5.3	流程图示例	7
6	异常捕捉与错误恢复	7
6.1	结果	8
7	自动测试	8
8	语法分析讨论：自顶向下 vs. 自底向上	8
8.1	技术的简单性	8
8.2	技术的通用性	9
8.3	语义动作表达能力	9
8.4	出错恢复	10
8.5	表格驱动方式	10
8.6	速度比较	10
9	实验感想	11

1 实验目的

利用 Java 语言手工编写一个 Oberon-0 语言的语法分析程序，该语法分析程序执行与实验三自动生成的语法分析程序类似的功能，生成一个流程图。

2 设计 Oberon-0 语言的翻译模式

消除左递归 左递归会导致递归下降无限循环因此需要消除，改造后产生时如代码 2 所示，可以看到对于 `const_decl_list`，可以通过循环实现。

```
const_part ::= CONST const_decl_list | /* empty */;  
const_decl_list ::= const_decl_list const_decl | /* empty */;  
const_decl ::= IDENTIFIER EQ expression SEMI;
```

Listing 1: 原产生式

```
const_part ::= CONST const_decl_list | /* empty */;  
const_decl_list ::= const_decl const_decl_list | /* empty */;  
const_decl ::= IDENTIFIER EQ expression SEMI;
```

Listing 2: 无左递归产生式

提取左公因子 递归下降根据向前看一个字符来进行展开，因此需要两个产生式的 first 集不相交。如代码 3 所示，`factor` 在看到 `IDENTIFIER` 时不知道展开哪一个产生式，而代码 4 提取左公因子后在 `selector_opt` 选择即可。

```
factor ::= IDENTIFIER selector | IDENTIFIER | ...;  
selector ::= DOT IDENTIFIER selector | LBRACKET expression  
           RBRACKET selector;
```

Listing 3: 原产生式

```
factor ::= IDENTIFIER selector_opt | ...;  
selector_opt ::= DOT IDENTIFIER selector | LBRACKET expression  
              RBRACKET selector | /* empty */;
```

Listing 4: 提取公因子产生式

具体文法定义见 `ex4/src/LLTable/grammar.txt` 所示。

3 FIRST / FOLLOW 集计算

计算 first 和 follow 集有助于递归下降程序的编写，因此写了一个 python 脚本计算该文法所有非终结符的 first 和 follow 集，具体见 `/ex4/src/LLTable/first.py`

First 计算方法

1. 若 X 为终结符，则 $\text{FIRST}(X) = \{X\}$;
2. 若 $X \rightarrow \varepsilon$ ，则 $\varepsilon \in \text{FIRST}(X)$;
3. 若 $X \rightarrow Y_1 Y_2 \cdots Y_k$:
 - 将 $\text{FIRST}(Y_1) \setminus \{\varepsilon\}$ 加入 $\text{FIRST}(X)$;
 - 若 $\varepsilon \in \text{FIRST}(Y_i)$ ($i < k$)，继续将 $\text{FIRST}(Y_{i+1}) \setminus \{\varepsilon\}$ 加入;
 - 若所有 Y_i 均可推导出 ε ，则 $\varepsilon \in \text{FIRST}(X)$ 。

Follow 计算方法

1. $\$ \in \text{FOLLOW}(S)$ (S 为开始符号);
2. 对产生式 $A \rightarrow \alpha B \beta$: 将 $\text{FIRST}(\beta) \setminus \{\varepsilon\}$ 加入 $\text{FOLLOW}(B)$;
3. 若 $A \rightarrow \alpha B$ 或 $A \rightarrow \alpha B \beta$ 且 $\varepsilon \in \text{FIRST}(\beta)$: 将 $\text{FOLLOW}(A)$ 加入 $\text{FOLLOW}(B)$ 。

对于 Follow 的第二条公式计算可以对每个产生式右部，从右往左扫一遍，维护一个 `right_first` 表示右侧的全部 First 集，当前为空串时，则将当前符号的 first 集与 `right_first` 取并集，否则直接为当前符号的 first 集，这样可以一遍扫描就找到每个非终结符的 Follow。当集合没有变化的时候退出循环。

4 递归下降分析器设计

4.1 整体架构

1. 非终结符对应子程序：每个非终结符对应一个 Java 方法;
2. 开始符号对应主程序：`module()` 是顶层入口;

3. **Lookahead 决定分支**: 用 `lookahead.getToken()` 进行预测, 向前看一个字符;
4. **属性传递**: 继承属性 $\rightarrow$ 形参; 综合属性 $\rightarrow$ 返回值; 子结点属性 $\rightarrow$ 局部变量;
5. **翻译模式**: 终结符匹配, 非终结符递归调用, 语义动作直接嵌入。

每次调用非终结符子程序, 会预先读一个 Token, 而每次从子程序返回他的 Follow Token。

非左递归的具体实现 以记录类型为例, 对应非终结符 `record_type`, 他有一个综合属性, 返回具体记录类型对象。首先分析 `field_list`, 并使用 `while` 循环遍历是否为 SEMI `field_list`, 最后跳过 END。

对于分析的语义动作, 直接嵌入到子程序中。

```
public mySymbol.RecordType record_type() {
    RecordType rt = new mySymbol.RecordType();
    lookahead = get_token(); // id
    fl = field_list();
    if (fl != null) rt.addField(fl);
    while (lookahead.getToken() == Token.SEMI) {
        lookahead = get_token();
        mySymbol.FpSection flNext = field_list();
        if (flNext != null) rt.addField(flNext);
    }
    if (lookahead.getToken() == Token.END) lookahead =
        get_token();
    return rt;
}
```

Listing 5: 记录类型分析

可为空串的非终结符 以实参类型为例, 他可以为空, 因此进入该子程序时, 先判断 `lookahead` 是否属于 `actual_parameters_opt` 的 Follow 集合, 如果是的话就展开为空串, 返回父程序继续分析。如代码 6 所示。

```
if(lookahead == Token.SEMI || lookahead == Token.END
|| lookahead == Token.ELSIF || lookahead == Token.ELSE) {
    return new ArrayList<>();
}
.... // 继续分析
```

Listing 6: 实参类型分析

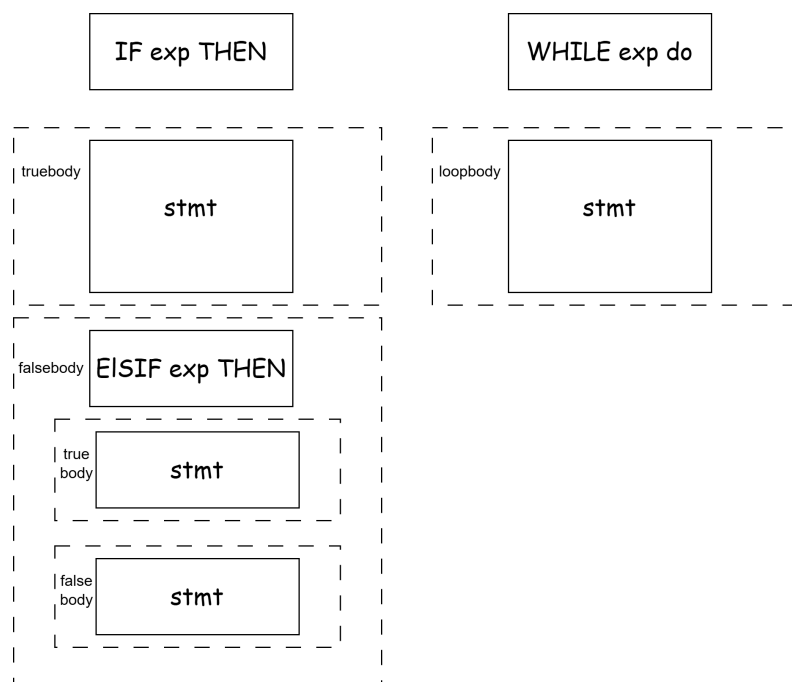


图 1: 条件及循环语句流程图生成

5 流程图生成

流程图生成是本分析器的核心翻译目标。分析器在语义分析阶段同步构建一棵流程图树，顶层对象为 `Module`，其下挂载若干 `Procedure`，每个 `Procedure` 包含一个 `StatementSequence`，语句之间支持嵌套，而为了支持 IF 和 WHILE 语句嵌套，flowchart 也给这两个语句增加了 `StatementSequence`，IF 提供 `trueBody` 和 `falseBody`，而 WHILE 提供 `loopBody`。

不同类型的语句由 `AbstractStatement` 抽象类管理。

5.0.1 语句序列栈

对于 Oberon-0 文法的 IF 和 WHILE 语句流程图分析如图 1 所示，而其中的 `stmt` 可以嵌入其他语句（包括 IF 和 WHILE 本身），从而能够实现嵌套。

分析器维护一个 `Deque<StatementSequence> seqStack` 表示当前执行的语句体：

- 进入 THEN/ELSIF/ELSE/DO 分支时，将对应的子 `StatementSequence` 压栈；
- 离开分支时弹栈；
- `addStmt()` 始终向栈顶序列追加，栈为空则追加到当前 `Procedure`。

```
private void if_statement() {
    // 跳过 IF
```

```
cond = expression();
ifStmt = new IfStatement(cond.getCode());
addStmt(ifStmt);           // 向当前活跃序列追加 IF 节点

// 跳过 THEN
seqStack.push(ifStmt.getTrueBody()); // 压入 trueBody
statement_sequence();
seqStack.pop();           // 弹出

// ELSIF 处理
IfStatement current = ifStmt;
while (lookahead == Token.ELSIF) {
    ...
    elsifStmt = new IfStatement(elsifCond.getCode());
    current.getFalseBody().add(elsifStmt);
    // 放入上一级 falseBody
    seqStack.push(elsifStmt.getTrueBody());
    statement_sequence();
    seqStack.pop();
    current = elsifStmt;
}

// ELSE 处理
if (lookahead.getToken() == Token.ELSE) {
    // 跳过 ELSE
    seqStack.push(current.getFalseBody()); // 压入 falseBody
    statement_sequence();
    seqStack.pop();
}
// 跳过 END
}
```

Listing 7: IF 语句中的压栈/弹栈

5.1 过程嵌套时的状态恢复

分析器会维护一个 Procedure 类型的 `currentProc` 维护当前属于哪一个过程。

对于过程嵌套时，每进入 `procedure_body` 时，需要先将当前状态保存，包括语句序列栈和当前过程，离开过程定义时，需要将语句序列栈恢复回调用前的状态，并将当前过程指向父过程。

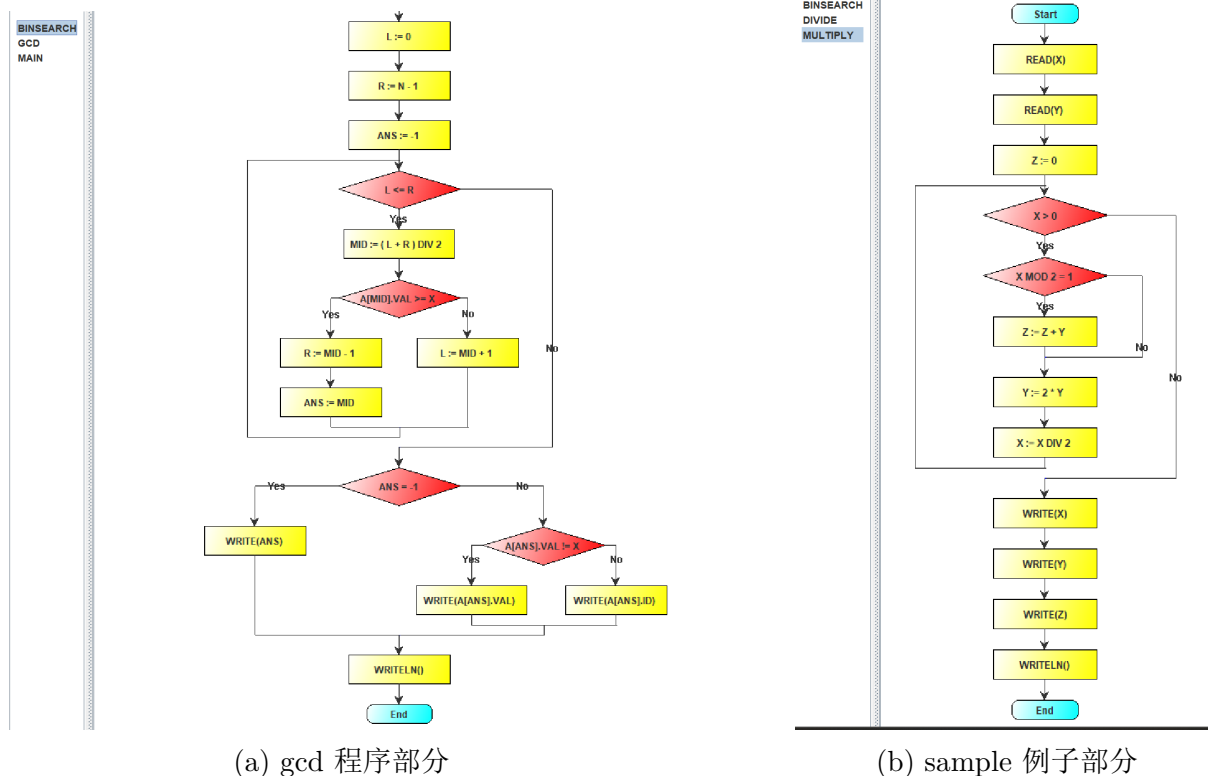


图 2: 流程图生成结果

不过 flowchart 不支持过程嵌套，因此不存储也行。

5.2 代码串生成

每个 Expression 对象携带一个 code 字符串，在递归构造过程中逐层拼接，最终成为流程图节点的标签文本。关系运算符使用 HTML 实体（<, >）以便在流程图中正确显示。

5.3 流程图示例

对于测试样例 gcd.obr 和文档中的 Sample 程序，流程图生成结果如 2 所示，可以看到对于复杂嵌套等生成正确。

6 异常捕捉与错误恢复

该部分与 lab3 较为相似，同样采用 `ErrorHandler.reportError()` 记录错误信息，然后根据上下文采取以下短路措施，使分析能继续推进。

语义分析包括类型匹配等与 lab3 完全一致。

缺失括号 比如形参声明缺失左括号, 可以查看 lookahead 是否属于 fp_section_list 的 First 集, 如果是的话就做缺少左括号处理。但是在表达式中, expression 的 First 集合和 factor 的 First 集合有交集, 因此较难判断此时缺失左括号的处理。

在应该匹配为右括号的地方没有右括号, 则缺失右括号。

缺失操作数 以 expression 为例, 分析完左 simple_expression 后, 如果 lookahead 属于 simple_expression 的 First 集合, 说明缺少操作符; 如果读取了操作符后, lookahead 属于 expression 的 Follow 集合, 说明缺少操作数。

6.1 结果

部分测例得到测试结果如图 3 所示。

```
===== Running Parser =====
MissingRightParenthesisException at line 39: Missing right parenthesis
Parsing completed in 0.688797 seconds.
===== Test Complete =====
```

(a) 缺失左括号

```
===== Running Parser =====
MissingOperatorException at line 33: Missing operator in I
MissingOperandException at line 62: Missing right operand for '&lt;'
TypeMismatchedException at line 62: Type mismatch in 'OR': expected BOOLEAN
TypeMismatchedException at line 62: Condition expression in IF statement must be BOOLEAN
Parsing completed in 0.5892976 seconds.
===== Test Complete =====
```

(b) 缺失操作符

```
===== Running Parser =====
TypeMismatchedException at line 33: For parameter "value", expected INTEGER, got NODE = RECORD
ID: INTEGER;
VAL: INTEGER;
END
SemanticException at line 33: Index selector applied to non-array type
TypeMismatchedException at line 33: For parameter "value", expected INTEGER, got ERROR
SemanticException at line 41: Undefined field "v"
TypeMismatchedException at line 41: Type mismatch in '&gt;': expected INTEGER
TypeMismatchedException at line 41: Condition expression in IF statement must be BOOLEAN
Parsing completed in 0.6771867 seconds.
===== Test Complete =====
```

(c) 类型不匹配

图 3: 异常测试结果

7 自动测试

使用与 lab3 同样的测试框架, 得到测试结果如图 4 所示。对于第八个测试, 由于递归下降向前看一个符号才能判断缺少右括号, 因此行数定位有点偏差。

8 语法分析讨论: 自顶向下 vs. 自底向上

8.1 技术的简单性

从实现难度来看, 使用 Java CUP 构造语法分析器通常更为简单。开发者只需根据文法编写产生式规则, 并在需要的位置添加语义动作, 分析器的状态机、分析表以及驱动程序均由工具自动生成, 因此能够减少大量底层实现工作。

```

===== Testing Parser =====

[FAIL] gcd008.obr
Expected line number(s) [52] not found in stderr. Actual lines: [53]
[PASS] gcd009.obr
[PASS] gcd010.obr
[PASS] gcd011.obr
[PASS] gcd012.obr
[PASS] gcd013.obr
[PASS] gcd014.obr
[PASS] gcd015.obr
[PASS] gcd016.obr
[PASS] gcd017.obr
[PASS] gcd018.obr
[PASS] gcd019.obr
[PASS] gcd020.obr
[PASS] gcd021.obr
[PASS] gcd022.obr

Results: 14 passed, 1 failed out of 15 tests.

=== Failure Summary ===
gcd008.obr: Expected line number(s) [52] not found in stderr. Actual lines: [53]

===== Test Complete =====

```

图 4: 批量测试结果

然而，从程序的可读性和调试难度来看，递归下降分析方法具有明显优势。递归下降分析器中的每个非终结符通常对应一个独立的递归函数，其程序结构与文法结构基本保持一致，便于理解和维护。当分析过程中出现错误时，可以通过设置断点、查看函数调用栈以及观察当前输入符号等方式快速定位问题。而 Java CUP 生成的分析器本质上是基于 LR 分析表驱动的状态机，其执行过程依赖于状态转移和移入、归约动作，程序逻辑相对隐蔽，调试时往往需要结合分析表和状态编号进行定位，因此调试难度较高。

8.2 技术的通用性

对于 LL 文法，设非终结符 (A) 有候选产生时为 $A \rightarrow \alpha_1 \mid \alpha_2$ ，需要满足：

- $FIRST(\alpha_1) \cap FIRST(\alpha_2) = \emptyset$
- 若 $\varepsilon \in FIRST(\alpha_1)$ ，则 $(FIRST(\alpha_2)) \cap FOLLOW(A) = \emptyset$

对于 LR 文法，不需要额外处理左递归等，文法范围明显大于 LL 文法。对于 CUP 使用的 LALR 是 LR 的减少状态版本，但支持的文法也大于 LL。

8.3 语义动作表达能力

递归下降分析器通常采用手工编写的方式实现，每个非终结符对应一个递归过程。语义动作可以直接嵌入到相应的分析函数中，并且能够方便地访问局部变量、返回值以及符号表等数据结构。因此，递归下降分析器在实现语法制导翻译时具有较高的灵活性，程序员可以根据需要在任意位置插入语义处理代码，使语法分析与语义分析紧密结合。

LR 分析器是自底向上的，因此对于综合属性的传递会更好，但是对于继承属性，由于移入时还没产生父节点，因此对继承属性的支持没有像递归下降一样灵活。

8.4 出错恢复

LL 分析和 LR 分析常用的出错恢复方法基本相同，主要包括恐慌模式和错误产生式两类。恐慌模式通过丢弃输入符号直至遇到同步符号（如分号、END 等）来恢复分析过程；错误产生式则通过为常见错误情况设计专门的产生式，从而识别并报告特定错误。

对于递归下降分析器，由于分析过程由程序员直接控制，因此可以在每个非终结符对应的分析函数中灵活地设计恢复策略。例如，可以根据 FOLLOW 集选择同步符号，也可以针对不同错误类型采取不同的恢复方式，因此实现较为灵活且易于定制。

对于 LR 分析器，错误恢复通常依赖分析表和状态机进行。以 Java CUP 为例，可以通过引入 `error` 符号以及相应的错误产生式实现自动恢复。虽然这种方式具有较好的通用性，但恢复过程与分析状态密切相关，调试和优化相对困难，恢复策略的可控性也不如手工实现的递归下降分析器。

8.5 表格驱动方式

令终结符个数为 N_t ，非终结符个数为 N_{nont} ，LR 的状态数为 N_s 。对于递归下降改为的表驱动分析器，其分析表大小为 $N_t \times (N_{nont} + 1)$ （包括结束符号）。而 LR 分析表大小为 $N_s \times (N_{nont} + N_t + 1)$ 。

由于 LR 分析表需要为每个状态维护 ACTION 和 GOTO 信息，其规模通常远大于 LL(1) 预测分析表。递归下降分析器甚至无需显式构造分析表，因此在空间开销方面具有明显优势。

8.6 速度比较

写一个 python 脚本生成语句嵌套深度为 250，表达式嵌套深度为 10，总共 20 个过程申明，每个过程 50 个句子的 Oberon-0 程序，具体见 `ex4/src/random_produce/random_p.py`。生成的代码大小为 12.3 MB。

对比 lab3 和 lab4 两种分析方法的运行时间，使用 `System.nanoTime()` 进行计时，得到结果如图 5 所示。可以看到递归下降方法竟然明显比 CUP 方法快很多。

Oberon-0 语法是非常适合递归下降的，他的大部分语句申明都在第一个字符。

```
(base) PS E:\github-res\tyylab3\ex4> run.bat
Active code page: 65001
===== Running Parser =====

Parsing completed in 0.4198557 seconds.

===== Test Complete =====
```

(a) 递归下降

```
(base) PS E:\github-res\tyylab3\ex3> run.bat
Active code page: 65001
===== Running Parser =====

Parsing and call graph construction completed in 2.9724725 seconds.

===== Test Complete =====
Press any key to continue . . .
```

(b) CUP LALR 分析

图 5: 计时

9 实验感想

本实验采用手工编写递归下降分析器的方式完成 Oberon-0 语言的语法分析，实现了流程图生成与错误恢复功能。通过与上一实验基于 Java CUP 的 LR 分析器进行比较，分析了两种方法在文法要求、实现复杂度、错误处理以及运行效率等方面的差异。实验表明，递归下降分析器结构清晰、便于实现语义处理和错误恢复，对于 Oberon-0 这类符合 LL(1) 条件的语言具有较高的分析效率；而 LR 分析器能够适用于更广泛的文法，但实现开销和运行开销相对较大。通过本实验进一步理解了两类语法分析技术的特点及其适用场景。